



Sky PAS

Security Review

Cantina Managed review by:
Christoph Michel, Lead Security Researcher
M4rio.eth, Lead Security Researcher

February 19, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Unlimited-to-finite rate-limit transition reverts due overflow, blocking decreases . . .	4
3.1.2	Initialize default <code>BeamState.maxChange</code> to 100% to unblock rate-limit decreases . .	4
3.1.3	Restrict <code>Configurator</code> updates to <code>BeamState</code> -defined rate-limit keys	4
3.2	Informational	5
3.2.1	Document forced unlimited rate limit behavior	5
3.2.2	Laniakea Docs <code>Configurator</code> inconsistencies	5
3.2.3	<code>addInitControllerActions</code> requires full call parameter specification leading to potential aBEAM operational overhead	6
3.2.4	Allow unlimited <code>maxAmount</code> regardless of <code>slope</code> in <code>Configurator</code> rate limit setup	6
3.2.5	Use <code>abi.encodeCall</code> for encoding payloads in <code>TimelockWrappers</code>	7
3.2.6	Revert when setting explicit default <code>rateLimits/controller</code> values	7
3.2.7	Validate <code>rateLimits_</code> and <code>controller</code> existence in <code>BeamState</code>	7
3.2.8	Minor issues, Typos & Documentation	8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Feb 9th to Feb 13th the Cantina team conducted a review of [pas](#) on commit hash [4fab3aac](#). The team identified a total of **11** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	2	1
Gas Optimizations	0	0	0
Informational	8	3	5
Total	11	5	6

The Cantina Managed team reviewed Sky's [pas](#) holistically on commit hash [027ea6d2](#) and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Low Risk

3.1.1 Unlimited-to-finite rate-limit transition reverts due overflow, blocking decreases

Severity: Low Risk

Context: `Configurator.sol#L105`

Description: `Configurator.setRateLimit` has a special branch for unlimited defaults (`defMaxAmount == type(uint256).max && defSlope == 0`) that writes unlimited data via `setUnlimitedRateLimitData`. If governance later changes defaults for the same key to finite values, subsequent calls go through the non-unlimited branch and evaluate:

```
current.maxAmount * maxChange / WAD
```

When `current.maxAmount == type(uint256).max` (from prior unlimited config), this multiplication overflows reverts before the `require` can evaluate. As a result, transitioning from unlimited to finite limits through `Configurator` is blocked and decreases should never block.

Recommendation: Handle unlimited current values before scaling:

- If `maxAmount <= current.maxAmount`, treat it as non-increase and skip scaling check.
- Only when `maxAmount > current.maxAmount`, run the current ceiling logic.

Sky: Fixed in commit `027ea6d2`.

Cantina Managed: Fix verified.

3.1.2 Initialize default `BeamState.maxChange` to 100% to unblock rate-limit decreases

Severity: Low Risk

Context: `Configurator.sol#L105`

Description: `BeamState.maxChange[address(0)]` (and `getMaxChange()` using this default) can currently resolve to `0`, which causes decrease calculations to multiply by `0` instead of allowing up to 100% movement.

```
require(maxAmount <= _max(current.maxAmount * maxChange / WAD, defMaxAmount),  
    ↪ "Configurator/exceeds-max-amount");
```

This will block decreases when setting a new value between `(defMaxAmount, current.maxAmount)` but decreases should never block. This scenario happens if either the `current.maxAmount` was `maxChange`-hop-increased above the `defMaxAmount`, or `defMaxAmount` was adjusted down by aBEAM afterwards.

Recommendation: Initialize `maxChange[address(0)]` to `WAD` in the `BeamState` constructor so the default is 100%. Since updates go through `setMaxChange` (which enforces `newValue >= WAD`), this preserves the invariant `maxChange[address(0)] >= WAD` and ensures `getMaxChange(*) >= WAD`, allowing the `_max` computation to work as expected.

Sky: This was solved differently in commit `027ea6d2`.

Cantina Managed: Fixed by decomposing `max` predicates into short-circuits and adding a check for `<= current` (this one fixes `maxChange == 0` bug described here). Setting `maxChange = 0` is explicitly allowed now in `BeamState`.

3.1.3 Restrict `Configurator` updates to `BeamState`-defined rate-limit keys

Severity: Low Risk

Context: `Configurator.sol#L108`

Description: The current `Configurator` logic allows an authorized `cBEAM msg.sender` to modify *any* key, including keys that were never explicitly initialized in `BeamState` (`initRateLimits[key][rateLimits].max/slope == 0`). This becomes risky when the key is

not init-ed but `current.maxAmount` is already non-zero, because the value can still be increased via `maxChange`. As a result, non-zero rate limits may be set or expanded effectively on keys that are not allow-listed.

Recommendation: Consider adding a guard that distinguishes between clearing and setting limits:

- Allow setting to 0 unconditionally (to remove limits).
- But require the key to be defined before allowing any non-zero update (e.g., `require(defMaxAmount > 0)` or equivalent explicit initialization check).

Sky: The idea is that the initial (`RateLimits`) state is thoroughly examined (and changed if needed), so we start in a position where it is determined fine that cBEAM can modify all existing non-zero keys.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Document forced unlimited rate limit behavior

Severity: Informational

Context: `Configurator.sol#L100`

Description: In case aBEAM has configured unlimited rate limits in `BeamState`, cBEAM is forced to indeed set unlimited rate limits behavior via `Configurator`. They cannot set a lower limit, or decide to later decrease the limit again, as unlimited rate limits are always enforced.

```
if (defMaxAmount == type(uint256).max && defSlope == 0) {  
    require(maxAmount == type(uint256).max && slope == 0,  
        "Configurator/unlimited-incorrect-params");  
}
```

Recommendation: Clarify and document this behavior.

Sky: The idea is mostly to differentiate the infinite value as something that limits the flexibility of the cBEAMS. For example it can be used for narrowing the cBEAMS ability to reduce withdraw limits (which might be important to stay high in case of emergencies). Added a comment in commit [027ea6d2](#).

Cantina Managed: Fix verified.

3.2.2 Laniakea Docs Configurator inconsistencies

Severity: Informational

Context: `configurator-unit.md`

Description: The following specifications in the Configurator smart contract docs do not match the current implementation.

1. `configurator-unit.md#L89`: Clarify role-to-capability mapping for Core Council, Mom, Freezer; Core Council cannot call `timeLock.pause()` unless granted `PAUSER_ROLE` (e.g. via `pausers[]`), while Mom can pause/stop via its path, and Freezer appears controller-side (`removeRelayer`) not `BeamState`-side.
2. `configurator-unit.md#L97`: Clarify the type of permission for the Grant cBEAM to GovOps, because in the docs is a Timelock-ed action but the implementation makes the grant operation IMMEDIATE `PASInit.sol#L111`.
3. `configurator-unit.md#L109`: Add caveat that decreasing rate limits does not apply for MAX init limits (intended for withdrawal-limit style keys).
4. `configurator-unit.md#L113`: Merge "Remove rate limit" with "Decrease rate limit" since removal is semantically decrease-to-zero.
5. `configurator-unit.md#L285`: `setPauContracts` is referenced in invariants, but that function does not exist in PAS as named.
6. `configurator-unit.md#L286`: "No configuration without init" is technically broken for existing non-zero `RateLimits` keys not present in `BeamState` init. See finding [Restrict Configurator updates to BeamState-defined rate-limit keys](#).

7. [configurator-unit.md#L288](#) increases can exceed `maxChange` while below init defaults. Suggested phrasing: increases **above init limits** are bounded by `maxChange` per hop.
8. [configurator-unit.md#L289](#): "Instant decreases are never blocked" needs caveats: MAX-init behavior (intended for withdrawal-like limits) and assumption that `BeamState` is not stopped.
9. [configurator-unit.md#L290](#): "Timelock on additions" is not fully given: While additions are delayed, `cBEAM` pairings (`setCBeamForController/RateLimits`) are immediate.
10. [configurator-unit.md#L301](#): `maxChange` is in WAD decimals, not basis points. It's also `100% + maxChange`, e.g., `1.2e18 = +20%`.
11. [configurator-unit.md#L321](#): Timelock delay parameter is not modified via delayed timelock path; it is admin-only via `updateDelayImmediately`.
12. [configurator-unit.md#L332](#): The current implementation differs from what is described here but allows for similar functionality.

Recommendation: Consider updating the documentation to match the implementation.

Sky: Acknowledged and passed to the stakeholders.

Cantina Managed: Acknowledged.

3.2.3 `addInitControllerActions` requires full call parameter specification leading to potential aBEAM operational overhead

Severity: Informational

Context: [BeamState.sol#L257](#)

Description: The data payloads in `BeamState.addInitControllerActions` currently require fully specified calldata and do not support wildcard/placeholder parameters. As a result, for calls like `abi.encodeCall(ControllerLike.grantRole, (RELAYER_ROLE, account))`, aBEAM/core council must predefine concrete values (e.g., `account`) up front rather than allowing cBEAM to select them later. This can lead to operational overhead as aBEAM is required for each specific init controller action through the Timelock.

Recommendation: Consider adding wildcard support for `addInitControllerActions` or explicit `Configurator` functions for common expected functions.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.2.4 Allow unlimited `maxAmount` regardless of `slope` in `Configurator` rate limit setup

Severity: Informational

Context: [Configurator.sol#L99](#)

Description: The unlimited rate limit path condition currently requires both `defMaxAmount == type(uint256).max` and `defSlope == 0`. However, `RateLimits.getCurrentRateLimit` treats a rate limit as unlimited based only on `maxAmount == type(uint256).max`, ignoring slope. This makes the setup logic more restrictive than runtime behavior and routes some unlimited configurations (with `slope > 0`) through the non-unlimited branch (including cooldown/hop handling), which is inconsistent.

Recommendation: Update the configurator logic so any `defMaxAmount == type(uint256).max` configuration is handled by the unlimited branch regardless of `defSlope`, basically ignoring `BeamState`'s slope in case `maxAmount` is max.

```
- if (defMaxAmount == type(uint256).max && defSlope == 0) {
+ if (defMaxAmount == type(uint256).max) {
```

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.2.5 Use `abi.encodeCall` for encoding payloads in `TimelockWrappers`

Severity: Informational

Context: `TimelockWrapperMainnet.sol#L159`

Description: The payload is currently encoded with `abi.encodeWithSelector`, which is more error-prone because argument ordering and types are not checked against the function signature at compile time.

Recommendation: Replace `abi.encodeWithSelector` with `abi.encodeCall` for all `BeamStateLike.*` payload construction, e.g.

```
abi.encodeCall(BeamStateLike.setHop, (rateLimits, hop))
```

to get compile-time validation of the target function and argument tuple.

The same issue applies to `TimelockWrapperForeign`.

Sky: Fixed in commit [207ef979](#).

Cantina Managed: Fix verified.

3.2.6 Revert when setting explicit default `rateLimits`/`controller` values

Severity: Informational

Context: `BeamState.sol#L185`

Description: `rateLimits_ == 0` and `controller == 0` is treated as the default/fallback behavior.

Recommendation: Consider reverting when explicitly registering the default values via `addRateLimits` / `addController` as these functions should be used for registering actual PAS contracts and not interfere with default semantics.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.2.7 Validate `rateLimits_` and `controller` existence in `BeamState`

Severity: Informational

Context: `BeamState.sol#L173`

Description: Some setters in `BeamState` do not validate that the `rateLimits_` or `controller` is valid. It should be either default `address(0)` or a registered one:

- `setHop`.
- `setMaxChange`.
- `addInitRateLimits`.
- `addInitControllerActions`.

Recommendation: Consider validating the parameter is either 0 or a registered `rateLimits_` / `controller` value.

```
require(rateLimits_ == 0 || rateLimits[rateLimits_] == 1,  
  ↳ "BeamState/not-existing-rateLimits");  
// or  
require(controller == 0 || controllers[controller] == 1,  
  ↳ "BeamState/not-existing-controller");
```

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.2.8 Minor issues, Typos & Documentation

Severity: Informational

Context: (See each case below)

Severity: Informational

Description:

`BeamState.sol#L108-L114`: Should be documented that these functions return the general value in case a value for the specific rate limit does not exist, same as we have on `getInitRateLimits`

- `smart-contracts/configurator-unit.md#role-capabilities`: The docs specify that the `Grant cBEAM to GovOps` is a Timelock-ed action but the implementation makes the grant operation IMMEDIATE `PASInit.sol#L111`
- `Configurator.sol#L122`: The `SetRateLimit` event should contain the `lastAmount`. Note that we do emit the `lastAmount` in the `RateLimitsLike(rateLimits).setRateLimitData` but that value can be truncated to `maxAmount` due to `RateLimitsLike(rateLimits).setRateLimitData(...,_min(maxAmount,lastAmount),...)`.

Recommendation: Consider fixing the issues mentioned above.

Sky:

- Comments added in this commit `027ea6d2`.
- Noted about the external repo documentation issue.
- Acknowledged about the event.

Cantina Managed: Fixed and acknowledged.